CSC 659-859

Spring 26

Lawrence Chiu,

Lchiu3@sfsu.edu,

3/14/26

Section 2: Audit of Original Database

Table 1. Summary statistics of the original dataset prior to splitting into training and verification databases. The table reports the number of samples, features, class distribution, missing values, and sample-to-feature ratio.

Metric	Value
Number of Samples	871
Number of Features	608
Class 0 Samples	781
Class 1 Samples	90
% Class 0	89.66705%
% Class 1	10.33295%
Missing Values	0
Sample-to-Feature Ratio	1.43257

The dataset contains 871 samples and 608 numerical features, representing gene expression levels. The final column, labeled *"Label"*, represents a binary class variable (0 or 1).

The original dataset was split into a Training Database and a Verification Database by randomly selecting one sample from each class for verification. The Training Database contains 869 samples with 608 features, while the Verification Database contains 2 samples with the same 608 features.

This section first describes the original source database; the required training and verification database statistics are summarized in Section 3 after the split

Description of Features and Data Format

The features correspond to gene expression measurements (e.g., GABRG2, CELF4, SRRM4, etc.), each represented as a continuous numerical value. The dataset is structured in tabular format, with each row representing one biological sample and each column representing a gene feature. The last column contains the binary class label.

Ground Truth and Class Labels

The class labels (0 and 1) were defined according to the referenced biological research study. These labels represent the established ground truth classification of each sample as defined in the original study.

Demography and Fairness

The dataset does not contain demographic variables such as age, gender, ethnicity, or geographic information. Therefore, demographic coverage and fairness cannot be directly assessed. Since only gene expression values are included, no demographic bias can be evaluated from this database.

Class Distribution and Imbalance

The class distribution is:

- Class 0: 781 samples (89.66705%)
- Class 1: 90 samples (10.33295%)

The minority class (Class 1) represents approximately 10.33% of total samples. Although slightly above the 10% threshold often used to define severe imbalance, the dataset should still be considered imbalanced. Therefore, evaluation metrics such as Precision, Recall, and F1-score will be particularly important during model assessment.

Feature Types

All 608 predictor variables are numerical continuous features (float64). The class label is categorical binary (0 or 1).

Missing Values

The dataset contains 0 missing values. All entries are present and valid. Zero values represent legitimate gene expression measurements and do not require imputation. Therefore, no preprocessing for missing data is necessary.

Sample-to-Feature Ratio

$$\text{Sample-to-feature ratio} = \frac{871}{608} = 1.43257$$

Ideally, the number of samples should be at least 10 times greater than the number of features to ensure strong generalization performance. In this database, the ratio is significantly lower than 10, indicating a high-dimensional setting. This increases the theoretical risk of overfitting for many machine learning algorithms. However, Random Forest is known to perform relatively well in high-dimensional spaces due to its ensemble structure and random feature selection mechanism.

Privacy and Personally Identifiable Information (PII)

The dataset contains only gene expression measurements and class labels. No personally identifiable information (PII) such as names, IDs, addresses, or demographic identifiers is included. Therefore, privacy risks appear minimal.

Conclusion of Audit

The dataset is high-dimensional and moderately imbalanced but contains no missing values and no privacy concerns. The structure and feature types are appropriate for Random Forest training. Careful evaluation using F1-score and confusion matrix analysis will be necessary due to class imbalance.

Section 3: Creation of Training DB and Verification DB

Two samples were randomly selected from the original database using a fixed random seed to ensure reproducibility. One sample from Class 1 and one sample from Class 0 were placed into a separate Verification Database. The remaining samples form the Training Database.

This ensures that the Verification Database contains one example from each class and remains completely unseen during model training.

Training Database Statistics

Table 2. Summary statistics of the Training Database after removing two samples for verification testing. This database is used for all Random Forest training, hyperparameter tuning, and accuracy estimation.

Metric	Value
Number of Samples	869
Number of Features	608
Class 0 Samples	780
Class 1 Samples	89
% Class 0	89.75834%
% Class 1	10.24166%
Missing Values	0

The Training Database remains slightly imbalanced, with Class 1 representing approximately 10.24% of samples. All features remain numerical and no missing values are present.

Verification Database Statistics

Table 3. Summary statistics of the Verification Database after removing two samples from the original dataset for final run-time testing. This database is reserved for the final RF prediction test and remains unseen during model training, hyperparameter tuning, and accuracy estimation.

Metric	Value
Number of Samples	2
Number of Features	608
Class 0 Samples	1
Class 1 Samples	1
% Class 0	50.0%
% Class 1	50.0%
Missing Values	0

The Verification Database contains exactly one sample from each class, ensuring balanced evaluation during the final run-time test.

Section 4 - SW Tools

Programming Language

- Python 3.13.7

Main ML Library

- Scikit-learn (RandomForestClassifier)

Supporting Libraries

- pandas (data handling)
- numpy (numerical operations)
- matplotlib (plotting)
- sklearn.metrics (confusion matrix, accuracy, etc.)

GenAI Tools

- ChatGPT 5.2

All experiments were conducted using the SciKit-learn implementation of Random Forest.

Section 5: Experimental Methods and Setup

The training data training_db.csv will be used only for all tuning/accuracy estimation (OOB + CV). Verification_db.csv will be untouched until Section 8 (run-time test).

Method 1: OOB Accuracy Estimation (Random Forest OOB)

Random Forest uses bootstrap sampling: each tree is trained on a bootstrap sample of the training set, leaving out ~36% of samples for that tree (out-of-bag samples). OOB accuracy is estimated by predicting each training sample using only trees for which that sample was out-of-bag, then computing accuracy from those OOB predictions. In SciKit this is returned as oob_score_ when oob_score = True.

For each training sample S_i (sample i), the algorithm identifies trees that did not use that sample during training. Only those trees are used to generate a prediction for that sample. The predicted class is compared with the true label to compute the prediction error. The final OOB error estimate is obtained by averaging the prediction errors over all training samples.

Because each tree evaluates samples that were not used during its training, OOB accuracy provides an estimate of model performance on unseen data that is approximately equivalent to cross-validation.

I will use the following SciKit tools/classes:

- sklearn.ensemble.RandomForestClassifier
- .fit(x_train, y_train) to train
- oob_score = True to enable OOB estimate
- Read clf.oob_score_ as OOB accuracy (SciKit gives OOB accuracy instead of error)

Hyperparameters + ranges I will use

- NTREE → n_estimators
- MTRY → max_features

Other RF params will remain default.

The dataset has **NF = 608 features**, so:

- $\sqrt{NF} \approx \sqrt{608} \approx 24.6 \rightarrow$ **25**

Recommended MTRY values (integers):

- $0.5\sqrt{NF} \approx 12$
- $\sqrt{NF} \approx 25$
- $2\sqrt{NF} \approx 50$

NTREE range:

- n_estimators: [500, 1000]

So Method 1 grid (example):

- n_estimators ∈ {500, 1000}
- max_features ∈ {12, 25, 50}

Fixed settings:

- bootstrap = True (default)
- oob_score = True
- random_state = 42 (reproducible)
- n_jobs = -1 (speed)

How I will compute OOB and other accuracy measures

OOB accuracy (Method 1 primary metric)
SciKit provides:

- OOB Accuracy = clf.oob_score_

To report OOB error:

- OOB Error = 1 − OOB Accuracy

In addition to OOB accuracy, OOB class predictions will be obtained from oob_decision_function_ by assigning each training sample to the class with the larger OOB vote. These OOB predictions will then be compared with the true labels to construct a full OOB confusion matrix. Accuracy, precision, recall, specificity, and F1 score for Method 1 will be computed directly from this OOB confusion matrix.

Equations	
Accuracy	$\frac{(TP + TN)}{(TP + TN + FP + FN)}$
Precision	$\frac{TP}{(TP + FP)}$
Recall	$\frac{TP}{(TP + FN)}$
Specificity	$\frac{TN}{(TN + FP)}$
F1	$\frac{2 \cdot TP}{(2 \cdot TP + FP + FN)}$

Method 2: 3-Fold Cross Validation

I will estimate RF performance on unseen data using **3-fold stratified cross-validation**. The training set is partitioned into 3 folds with similar class proportions. For each fold k:

- Train RF on the other 2 folds (training portion)
- Test on the held-out fold (test portion)
- Compute confusion matrix and accuracy metrics
  After running all 3 folds, sum confusion matrices across folds to create a final combined confusion matrix and compute overall accuracy/precision/recall/F1.

Fold 1

1. Build train/test indices for fold 1 (stratified)
2. Train RF on fold1-train
3. Predict fold1-test → compute confusion matrix + metrics

Fold 2

Repeat with fold 2 as test fold.

Fold 3

Repeat with fold 3 as test fold.

I will use StratifiedKFold(n_splits=3, shuffle=True, random_state=42) only to generate the training and testing indices for each fold. I will then manually perform the cross-validation process by looping through the folds and applying .fit() and .predict() within each iteration. Stratification ensures that each fold maintains approximately the same class distribution as the full training dataset.

SciKit tools/classes that will be used:

- sklearn.model_selection.StratifiedKFold (to generate folds)
- sklearn.ensemble.RandomForestClassifier (train per fold)
- sklearn.metrics.confusion_matrix

Metrics to Compute in CV

Per fold:

- Confusion matrix (TP/TN/FP/FN)
- Accuracy (required)
- F1

Final CV metrics:

- Combine folds by summing the 3 confusion matrices element-wise
- Compute overall accuracy/precision/recall/F1 from the combined matrix

Comparison Strategy

I will compare:

- Method 1: OOB accuracy (from oob_score_)
- Method 2: 3-fold CV accuracy (from combined confusion matrix)

Expectation:

OOB and CV accuracy should be close (small difference), since both estimate future performance.

I will present the comparison later in Section 6 as a small table:

- OOB accuracy
- CV accuracy
- absolute difference

Section 6:

Method 1 Results (OOB)

Table 4. Out-of-bag (OOB) accuracy results for Random Forest models trained on the Training Database using different combinations of number of trees (NTREE) and number of features tested at each split (MTRY)

Trees	Features per split	OOB Accuracy
500	12	0.97928
500	25	0.97928
500	50	0.98619
1000	12	0.97928
1000	25	0.98273
1000	50	0.98388

The Random Forest model was trained using the hyperparameter grid described in Section 5. The best performing configuration was:

- n_estimators = 500
- max_features = 50

The corresponding performance was:

- OOB Accuracy = 0.98619
- OOB Error = 0.01381

Table 5. Out-of-bag (OOB) confusion matrix for the selected Random Forest model trained on the Training Database using n_estimators = 500, max_features = 50, bootstrap = True, oob_score = True, and random_state = 42. Rows represent true class labels and columns represent OOB-classified labels. Full sample counts are shown.

True / Classified	Classified 0	Classified 1
True 0	779	1
True 1	11	78

The OOB confusion matrix shows that 779 Class 0 samples and 78 Class 1 samples were correctly classified, while 1 Class 0 sample was misclassified as Class 1 and 11 Class 1 samples were misclassified as Class 0.

These OOB class decisions were obtained from the Random Forest model's out-of-bag predictions on the Training Database and were used to compute the Method 1 accuracy measures reported in Table 6.

Table 6. Out-of-bag (OOB) performance measures for the selected Random Forest model trained on the Training Database using n_estimators = 500, max_features = 50, bootstrap = True, oob_score = True, random_state = 42, and n_jobs = -1. Reported measures include OOB accuracy, OOB error, precision, recall, specificity, and F1 score, all computed from the OOB confusion matrix in Table 5.

Metric	**Value**
OOB Accuracy	0.98619
OOB Error	0.01381
Precision	0.98734
Recall	0.87640
Specificity	0.99872
F1	0.92857

Table 7. Results for the 3-fold stratified cross-validation experiment on the Training Database using the selected Random Forest configuration (n_estimators = 500, max_features = 50, random_state = 42, n_jobs = -1). For each fold, the table reports the number of Class 0 and Class 1 samples in the training and test partitions, together with the held-out fold accuracy and F1 score computed from predictions on the test partition.

Fold	Train Class 0	Train Class 1	Test Class 0	Test Class 1	Accuracy	F1
1	520	59	260	30	0.97586	0.86792
2	520	59	260	30	0.98966	0.94915
3	520	60	260	29	0.98616	0.92593

This configuration was selected as the final model for cross-validation and run-time testing.

Combined Confusion Matrix

Table 8. Combined confusion matrix obtained from 3-fold stratified cross-validation on the Training Database using the selected Random Forest configuration (n_estimators = 500, max_features = 50, random_state = 42, n_jobs = -1). Rows represent true class labels and columns represent predicted class labels. Full sample counts are shown after summing the confusion matrices from the three held-out test folds.

True / Classified	Classified 0	Classified 1
True 0	779	1
True 1	13	76

The final combined confusion matrix was obtained by summing the corresponding quadrants from the three fold confusion matrices. Final accuracy measures were then computed from this combined confusion matrix, so that they reflect performance across the entire Training Database. This follows the procedure described in the lecture slides

The following accuracy measures were computed directly from the combined confusion matrix shown in Table 8.

Metrics from the Confusion Matrix

Table 9. Accuracy measures computed from the combined confusion matrix obtained from 3-fold stratified cross-validation on the Training Database using the selected Random Forest configuration (n_estimators = 500, max_features = 50, random_state = 42, n_jobs = -1). Reported measures include accuracy, precision, recall, specificity, and F1 score, all computed from the summed confusion matrix across the three test folds.

Accuracy	0.98389
Precision	0.98701
Recall	0.85393
Specificity	0.99872
F1	0.91566

The model demonstrates very high overall accuracy and specificity. However, recall for the minority class is lower due to class imbalance.

Comparison of Method 1 and Method 2

Table 10. Comparison of Method 1 and Method 2 accuracy estimates for the selected Random Forest configuration. Shown are OOB accuracy, 3-fold CV accuracy, and the absolute difference between the two values.

Metric	Value
OOB Accuracy	0.98619
CV Accuracy	0.98389
Absolute Difference	0.00230

The OOB and CV accuracy values are very close, differing by only 0.00230. This indicates that OOB provides a reliable estimate of future model performance.

Section 7: Feature Ranking

Top 10 Most Important Features

Table 11. Top 10 features ranked by Gini importance from the best trained Random Forest model fitted on the full Training Database using n_estimators = 500, max_features = 50, random_state = 42, and n_jobs = -1. Higher importance values indicate greater contribution of a feature to the model's classification decisions.

Feature	Importance
COL5A2.1	0.08678
COL5A2	0.07519
COL5A2.2	0.07406
NDNF.2	0.04245
FAT1	0.03772
NDNF	0.03674
NDNF.1	0.03349
SST	0.02768
NPNT	0.02497
TOX3.1	0.02038

Several of the highest-ranked genes correspond directly to those highlighted in the referenced biological study, including **COL5A2**, **NDNF**, and **FAT1**. Notably, COL5A2 appears multiple times among the top-ranked features, indicating strong predictive influence. NDNF also appears repeatedly in the top positions.

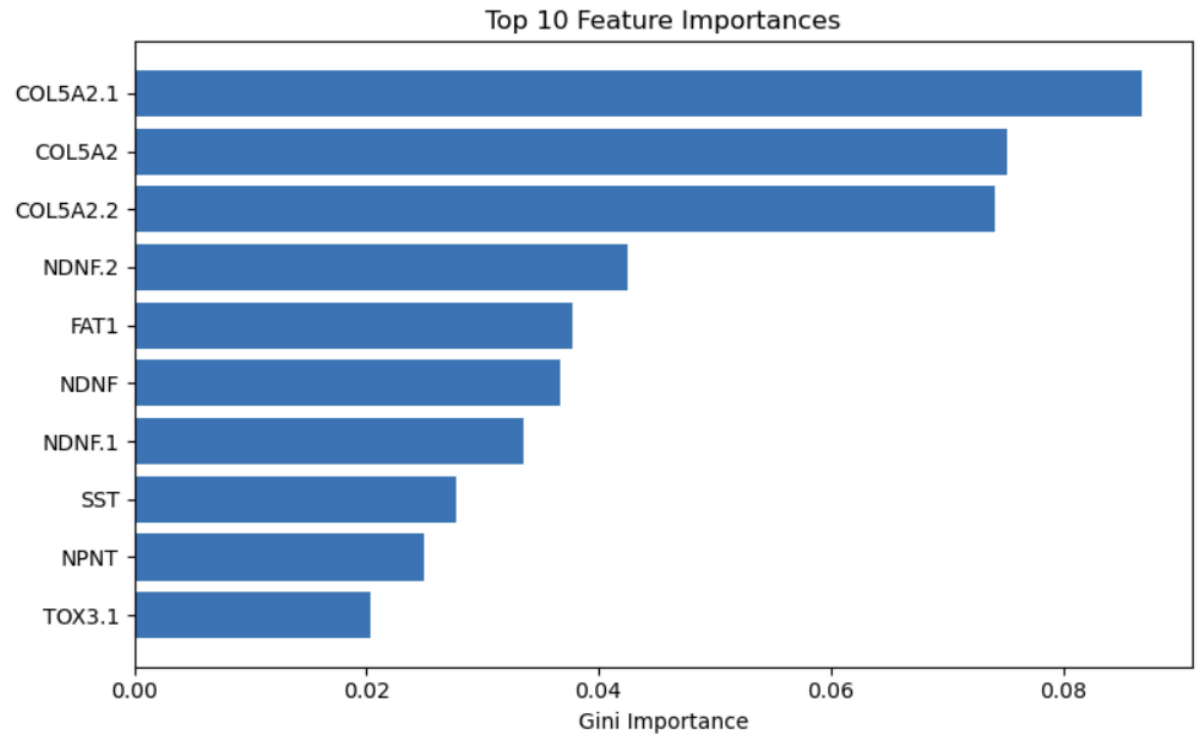
The importance values decrease substantially after the first few genes, suggesting that a relatively small subset of features contributes most of the predictive power. This pattern is common in high-dimensional gene expression datasets, where only a limited number of genes drive classification performance.

Although feature importance rankings may vary slightly depending on hyperparameters and random seed due to correlated gene expression patterns, the most predictive genes remain consistently near the top.

These results demonstrate that the Random Forest model identifies biologically meaningful predictors consistent with prior research.

The sharp decline in importance after the top few genes suggests that dimensionality reduction or recursive feature elimination may be possible without substantial loss of predictive performance. Based on the sharp drop in Gini importance after the first several genes, I would suggest starting with approximately the top 5 to 10 ranked features in a production setting, then validating whether accuracy remains acceptably close to the full-feature model.

Figure 1. Bar chart of the top 10 features ranked by Gini importance from the best trained Random Forest model fitted on the full Training Database using n_estimators = 500, max_features = 50, random_state = 42, and n_jobs = -1. The x-axis shows Gini importance and the y-axis shows feature names. Higher values indicate greater contribution of a feature to the model's classification decisions.



Code Snippet for Bar table with Consultation with ChatGPT 5.2

```
import matplotlib.pyplot as plt
import pandas as pd

importances = best_model.feature_importances_
feature_names = training_df.drop("Label", axis=1).columns

feat_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

feat_df.head(10)

top10 = feat_df.head(10)

plt.figure(figsize=(8,5))
plt.barh(top10["Feature"][::-1], top10["Importance"][::-1])
plt.xlabel("Gini Importance")
plt.title("Top 10 Feature Importances")
plt.tight_layout()
plt.show()
```

Section 8: RF Run Time Test

The final Random Forest model (n_estimators = 500, max_features = 50) was trained using the full Training Database and applied to the Verification Database.

Code Snippet with Consultation with ChatGPT 5.2:

```
from sklearn.ensemble import RandomForestClassifier

# Prepare training data
x_train = training_df.drop("Label", axis=1).values
y_train = training_df["Label"].values

# Train final model using our best hyperparameters
final_model = RandomForestClassifier(
    n_estimators=500,
    max_features=50,
    random_state=42,
    n_jobs=-1
)

final_model.fit(x_train, y_train)

# Prepare verification data
x_ver = verification_df.drop("Label", axis=1).values
y_ver = verification_df["Label"].values

# Predict class labels
y_pred_ver = final_model.predict(x_ver)

# Predict class probabilities
y_prob_ver = final_model.predict_proba(x_ver)

y_ver, y_pred_ver, y_prob_ver
```

Prediction Results

Table 12. Run-time prediction results for the two Verification Database samples using the final Random Forest model trained on the full Training Database using n_estimators = 500, max_features = 50, random_state = 42, and n_jobs = -1. Shown for each sample are the true label, predicted label, and predicted class probabilities returned by predict_proba().

Sample	True Label	Predicted Label	Prob(Class 0)	Prob(Class 1)
1	1	1	0.142	0.858
2	0	0	0.972	0.028

Both verification samples were correctly classified. Sample 1 was predicted as Class 1 with probability 0.858, indicating that 85.8% of trees voted for Class 1. Sample 2 was predicted as Class 0 with probability 0.972, indicating strong agreement among trees. Model confidence was established using the class probabilities returned by the predict_proba() function, which reflects the proportion of trees voting for each class. The high probabilities indicate strong predictive confidence for both samples.

Section 9: References and Resources

Aevermann B., Novotny M., Bakken T., Miller J., Diehl A., Osumi-Sutherland D., Lasken R., Lein E., Scheuermann R.: "Cell type discovery using single cell transcriptomics: implications for ontological representation", Human Molecular Genetics 27(R1): R40-R47 · March 2018

Slides from:
Intro to RF with Scikit-Learn from former student V2 03-08-20
RF Experiments - Useful hints for SciKit users

ChatGPT 5.2

Appendix I: Use of GenAI

The GenAI tool used for assistance/consultation/code generation during the writing of this paper was **ChatGPT 5.2**.

ChatGPT 5.2 was used for several tasks such as:

1. Providing a clear walkthrough of the assignment in a digestible manner.
2. Assistance with Section 2's Audit of Database and generation of code for "Section 2: Audit of Original Database" with table calculations.
3. Creation of Section 3's training and verification DB.
4. Assistance with the entirety of Section 5.
5. Generation of code for Section 6's results.
6. Consultation with results from Section 7's Feature Importance Analysis

7. Generation of the code to plot a Bar Graph of the data “Top 10 Feature Importances.”
8. Generation of the code for “Section 8: RF Run Time Test” to produce prediction results.

How ChatGPT 5.2 helped:

1. ChatGPT helped with breaking down concepts into something much more understandable, and then scaling up from there. I understand that the slides have clearly laid out the steps for each section, however, I do very much like the way ChatGPT is capable of breaking it down further (as well as the ability to put things in “Layman’s Terms”) as it helps my understanding for that section and I am personally easily overwhelmed. For every section listed in homework 2, I would prompt ChatGPT to break it down further (such as smaller steps) and have it explain why. For instance: I prompted ChatGPT with Method 1 and 2 to help break it down to smaller steps, have it explain the reasoning behind their response, and then applied it to this paper. I would rate this use of ChatGPT a 5/5 as it was extremely helpful in slowly walking through the assignment in a digestible manner.
2. “Section 2: Audit of Original Database” was assisted by ChatGPT in auditing the given database. This was through a step-by-step process of referring to previous lecture slides, asking for ChatGPT’s input, coming up with my original audit, and lastly having ChatGPT refine it further. The code was also generated by ChatGPT to display the statistics of the original database. I would rate my use of ChatGPT here a 5/5 as it helped refine my original answers and assisted in the audit of the original database.
3. The code to separate the original database into both the training database and the verification database was generated by ChatGPT. Not only that, but the code to calculate the statistics for both databases was generated by ChatGPT. I would rate this use of ChatGPT a 5/5 as it was simply a convenient way of quickly getting the two databases.
4. ChatGPT was used in the entirety of “Section 5: Experimental Methods and Setup.” ChatGPT assisted in explaining Method 1: OOB accuracy estimation and suggestion of parameters and ranges of hyper-parameters that would be used. Not only that, but it was also used for Method 2: 3-Fold Cross Validation with steps included. The main assistance from ChatGPT was finding the functions to achieve 3 folds manually. I would rate this use of ChatGPT a 4/5 as it was useful but it was having trouble providing a valid reason why these values should be used. However, it did provide me with a reasonable explanation in the end and also gave a great breakdown on Random Forest which I used in Section 5.
5. Section 6’s results were generated by ChatGPT. This would include both the generation of the code to compute results, but also ChatGPT was also consulted with and helped assist with explanation, my combined confusion matrix, and the metrics for the aforementioned confusion matrix. The use of ChatGPT would be 5/5 as it assisted in the majority of this section, including the comparison of Method 1 and Method 2 and helped further refine explanations.

6. Section 7's feature rankings were generated by ChatGPT (i.e. the code). Analysis of the feature rankings were also assisted/consulted with ChatGPT. This would get a 5/5 in terms of usefulness as it generally made the process much simpler.
7. The code to create the bar graph presented in Section 7 was generated by ChatGPT. The code snippet is included. I would rate this a 5/5 usage of ChatGPT since it achieved it's purpose.
8. ChatGPT was used to generate the prediction results for Section 8 as well as assist with the analysis of the prediction results. 5/5 usage of ChatGPT as it proved to be beneficial. The code snippet is also provided in the paper.

My only major prompt would be:
Prompt #1:

> *"I will list out requirements for a major assignment that would be a Machine Learning pipeline experiment with Random Forest ML (RF) using SciKit SW Toolkits.*
> *Here is my objective:*
> *Audit and import a given training database*
> *Create a training a verification DB from the given DB*
> *Train random forest and choose the optimal RF using Out-of-bag (OOB) and CV methods, compute accuracy measures from each and compare*
> *Compute random forest feature ranking and discuss explainability with regards to ground truth*
> *Apply run time (optimal) random forest in prediction mode on verification DB*
> *Present key results in an easy to read way with full data provenance.*
> *Check, analyze, and evaluate results*
>
> *These will be numbered in Sections given by me. Please outline and explain in as simple of a manner as possible, with an explanation using Layman's Terms on the side."*

Most, if not, all of my following prompts would always be formatted as:

> *"For Section [Section #] I have to [Section information], please break it down and explain the purpose for each step in as simple of a manner as possible."*

The rest of my prompts would be smaller prompts which would usually be a question, clarification, or the creation of code for data/table.
(i.e. *Generate code to display a bar graph of the Top 10 Feature Importances assuming the use of Scikit*)

Appendix II: Code

```
# CSC 659-859 Spring 2026 HW 2
# Purpose: Random Forest ML pipeline experiment on i1 cluster data
# Author: Lawrence Chiu
# Date: 3/14/26
```

```python
# Section 2: Audit of Database
import pandas as pd

# Load the original dataset (renamed since original did not work)
df = pd.read_csv("i1_cluster.csv")

# Number of samples
num_samples = df.shape[0]

# Number of features (excluding the label column)
num_features = df.shape[1] - 1

# Class distribution
class_counts = df["Label"].value_counts()

# Class percentages
class_percent = df["Label"].value_counts(normalize=True) * 100

# Missing values
missing_values = df.isnull().sum().sum()

# Sample-to-feature ratio
sample_feature_ratio = num_samples / num_features

print("Number of Samples:", num_samples)
print("Number of Features:", num_features)
print("Class 0 Samples:", class_counts[0])
print("Class 1 Samples:", class_counts[1])
print("% Class 0:", class_percent[0])
print("% Class 1:", class_percent[1])
print("Missing Values:", missing_values)
print("Sample-to-Feature Ratio:", sample_feature_ratio)
```

```python
# Section 3: Creating Training and Verification DB
import pandas as pd
import numpy as np

# Load original dataset
df = pd.read_csv("i1_cluster.csv")
```

```
# Fix random seed for reproducibility
np.random.seed(42)

# Select one sample from each class
class1_sample = df[df["Label"] == 1].sample(n=1)
class0_sample = df[df["Label"] == 0].sample(n=1)

# Create verification database
verification_df = pd.concat([class1_sample, class0_sample])

# Create training database (remove those two samples)
training_df = df.drop(verification_df.index)
```

```
# Section 3: Generate Training Database Statistics
train_samples = training_df.shape[0]
train_features = training_df.shape[1] - 1

train_class_counts = training_df["Label"].value_counts()
train_class_percent = training_df["Label"].value_counts(normalize=True) *
100

train_missing = training_df.isnull().sum().sum()

print("Number of Samples:", train_samples)
print("Number of Features:", train_features)
print("Class 0 Samples:", train_class_counts[0])
print("Class 1 Samples:", train_class_counts[1])
print("% Class 0:", train_class_percent[0])
print("% Class 1:", train_class_percent[1])
print("Missing Values:", train_missing)
```

```
# Section 6: Method 1 Results (OOB)
from sklearn.ensemble import RandomForestClassifier
# Separate features and label
x_train = training_df.drop("Label", axis=1).values
y_train = training_df["Label"].values
results = []

for n in [500, 1000]:
    for m in [12, 25, 50]:
```

```
        clf = RandomForestClassifier(
            n_estimators=n,
            max_features=m,
            oob_score=True,
            bootstrap=True,
            random_state=42,
            n_jobs=-1
        )
        clf.fit(x_train, y_train)

        oob_acc = clf.oob_score_

        results.append((n, m, oob_acc))

results
```

```
# Section 6: Method 1 - OOB confusion matrix and full accuracy measures
from ChatGPT
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
import pandas as pd
import numpy as np

# Use the same final Method 1 settings you report in the paper
BEST_NTREE = 500
BEST_MTRY = 50

# Training data
X = training_df.drop("Label", axis=1).values
y = training_df["Label"].values

# Train RF with OOB enabled
oob_model = RandomForestClassifier(
    n_estimators=BEST_NTREE,
    max_features=BEST_MTRY,
    bootstrap=True,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)
```

```
oob_model.fit(X, y)

# OOB predicted class probabilities for each training sample
# IMPORTANT:
# Use argmax across the 2 class columns to match scikit's OOB class
decision.
oob_prob = oob_model.oob_decision_function_
oob_pred = np.argmax(oob_prob, axis=1)

# OOB confusion matrix
# Format with labels=[0,1]:
# [[TN, FP],
#  [FN, TP]]
cm_oob = confusion_matrix(y, oob_pred, labels=[0, 1])

TN, FP = cm_oob[0, 0], cm_oob[0, 1]
FN, TP = cm_oob[1, 0], cm_oob[1, 1]
total = TN + FP + FN + TP

# Accuracy measures
oob_accuracy = (TP + TN) / total if total else 0.0
oob_precision = TP / (TP + FP) if (TP + FP) else 0.0
oob_recall = TP / (TP + FN) if (TP + FN) else 0.0
oob_specificity = TN / (TN + FP) if (TN + FP) else 0.0
oob_f1 = (2 * TP) / (2 * TP + FP + FN) if (2 * TP + FP + FN) else 0.0
oob_error = 1 - oob_model.oob_score_

# Confusion matrix table for report
oob_cm_table = pd.DataFrame(
    [[TN, FP],
     [FN, TP]],
    index=["True 0", "True 1"],
    columns=["Classified 0", "Classified 1"]
)

# Accuracy table for report
oob_metrics_table = pd.DataFrame({
    "Metric": ["OOB Accuracy", "OOB Error", "Precision", "Recall",
"Specificity", "F1"],
    "Value": [
        f"{oob_model.oob_score_:.5f}",
        f"{oob_error:.5f}",
```

```
        f"{oob_precision:.5f}",
        f"{oob_recall:.5f}",
        f"{oob_specificity:.5f}",
        f"{oob_f1:.5f}"
    ]
})

print("\nMethod 1 OOB confusion matrix:\n")
print(oob_cm_table.to_string())

print("\nMethod 1 OOB accuracy table:\n")
print(oob_metrics_table.to_string(index=False))

print("\nMethod 1 OOB confusion matrix (Markdown):\n")
print(oob_cm_table.to_markdown())

print("\nMethod 1 OOB metrics table (Markdown):\n")
print(oob_metrics_table.to_markdown(index=False))

print("\nMethod 1 OOB confusion matrix (LaTeX):\n")
print(oob_cm_table.to_latex())

print("\nMethod 1 OOB metrics table (LaTeX):\n")
print(oob_metrics_table.to_latex(index=False))
```

```
# Section 6: Method 2 (3-Fold CV) fold summary table with ChatGPT 5.2
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
import pandas as pd
import numpy as np

# Best RF settings selected from Method 1
BEST_NTREE = 500
BEST_MTRY = 50

# Prepare X and y
X = training_df.drop("Label", axis=1).values
y = training_df["Label"].values

# 3-fold stratified CV
```

```
skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

fold_rows = []
cm_total = np.zeros((2, 2), dtype=int)

for fold_id, (tr_idx, te_idx) in enumerate(skf.split(X, y), start=1):
    X_tr, X_te = X[tr_idx], X[te_idx]
    y_tr, y_te = y[tr_idx], y[te_idx]

    clf = RandomForestClassifier(
        n_estimators=BEST_NTREE,
        max_features=BEST_MTRY,
        random_state=42,
        n_jobs=-1
    )

    clf.fit(X_tr, y_tr)
    y_pred = clf.predict(X_te)

    # Confusion matrix in format:
    # [[TN, FP],
    #  [FN, TP]]
    cm = confusion_matrix(y_te, y_pred, labels=[0, 1])
    cm_total += cm

    # Class counts for this fold
    train_class0 = int((y_tr == 0).sum())
    train_class1 = int((y_tr == 1).sum())
    test_class0  = int((y_te == 0).sum())
    test_class1  = int((y_te == 1).sum())

    TN, FP = cm[0]
    FN, TP = cm[1]

    total = TN + FP + FN + TP
    accuracy = (TP + TN) / total if total else 0.0
    f1 = (2 * TP) / (2 * TP + FP + FN) if (2 * TP + FP + FN) else 0.0

    fold_rows.append({
        "Fold": fold_id,
        "Train Class 0": train_class0,
        "Train Class 1": train_class1,
```

```
        "Test Class 0": test_class0,
        "Test Class 1": test_class1,
        "Accuracy": f"{accuracy:.5f}",
        "F1": f"{f1:.5f}"
    })

fold_table = pd.DataFrame(fold_rows)

print("\nFold summary table:\n")
print(fold_table.to_string(index=False))

print("\nMarkdown version:\n")
print(fold_table.to_markdown(index=False))

print("\nLaTeX version:\n")
print(fold_table.to_latex(index=False, escape=False))
```

```
# Section 6 Metrics from the Confusion Matrix
def metrics_from_cm(cm):
    # cm format (with labels=[0,1]):
    # [[TN, FP],
    #  [FN, TP]]
    TN, FP = cm[0, 0], cm[0, 1]
    FN, TP = cm[1, 0], cm[1, 1]
    total = TN + FP + FN + TP

    acc = (TP + TN) / total if total else 0.0
    prec = TP / (TP + FP) if (TP + FP) else 0.0
    rec = TP / (TP + FN) if (TP + FN) else 0.0
    spec = TN / (TN + FP) if (TN + FP) else 0.0
    f1 = (2 * TP) / (2 * TP + FP + FN) if (2 * TP + FP + FN) else 0.0

    return acc, prec, rec, spec, f1

acc, prec, rec, spec, f1 = metrics_from_cm(cm_total)
acc, prec, rec, spec, f1
```

```
# Section 6 Best OOB accuracy from Method 1
oob_acc = 0.98619

# CV accuracy from Method 2
```

```
cv_acc = round(acc, 5)

# Absolute difference
abs_diff = round(abs(oob_acc - cv_acc), 5)

comparison_table = pd.DataFrame({
"Metric": ["OOB Accuracy", "CV Accuracy", "Absolute Difference"],
"Value": [round(oob_acc, 5), cv_acc, abs_diff]
})

comparison_table
```

```
# Section 7: Feature Ranking
best_model = RandomForestClassifier(
    n_estimators=500,
    max_features=50,
    random_state=42,
    n_jobs=-1
)

best_model.fit(x_train, y_train)

importances = best_model.feature_importances_
feature_names = training_df.drop("Label", axis=1).columns

feat_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

feat_df.head(10)
```

```
# Section 7 Bar Graph
import matplotlib.pyplot as plt
import pandas as pd

importances = best_model.feature_importances_
feature_names = training_df.drop("Label", axis=1).columns

feat_df = pd.DataFrame({
    "Feature": feature_names,
```

```
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

feat_df.head(10)

top10 = feat_df.head(10)

plt.figure(figsize=(8,5))
plt.barh(top10["Feature"][::-1], top10["Importance"][::-1])
plt.xlabel("Gini Importance")
plt.title("Top 10 Feature Importances")
plt.tight_layout()
plt.show()
```

```
# Section 8 Prediction Results
from sklearn.ensemble import RandomForestClassifier

# Prepare training data
x_train = training_df.drop("Label", axis=1).values
y_train = training_df["Label"].values

# Train final model using best hyperparameters
final_model = RandomForestClassifier(
    n_estimators=500,
    max_features=50,
    random_state=42,
    n_jobs=-1
)

final_model.fit(x_train, y_train)

# Prepare verification data
x_ver = verification_df.drop("Label", axis=1).values
y_ver = verification_df["Label"].values

# Predict class labels
y_pred_ver = final_model.predict(x_ver)

# Predict class probabilities
y_prob_ver = final_model.predict_proba(x_ver)
```

```
y_ver, y_pred_ver, y_prob_ver
```